

Fundamentals of Data Science

Volume 2: Reference and Solutions

Mohammad Sabokrou

Table of Contents

How to use this volume

This volume is not meant to be read through. It is meant to sit open beside you.

Part	What is in it	When you want it
1	Model answers to all 60 review questions	After you have written your own answer
2	Code cookbook, organised by task	Whenever you cannot remember the syntax
3	Formula reference	Checking an equation
4	Decision tables	Choosing a summary, chart, metric, or model
5	Every common-mistakes table in one place	Debugging, and before submitting
6	Glossary	Meeting a term you have forgotten
7	The dataset	Checking what a column means
8	Responsible-practice checklist	Before delivering anything
9	References and further study	Going deeper

One thing about Part 1

Model answers are in this volume rather than beside the questions on purpose.

Write your own answer first, in full, before you look. Comparing a finished attempt against a model answer tells you exactly where your understanding diverges. Reading the model answer cold tells you only that it sounds reasonable — which it will, and which teaches you nothing.

Most model answers end with a **common error**. That paragraph is often more useful than the answer itself, because it names the mistake that most people make on that specific question, and there is a good chance it is yours.

Part 1 — Model answers

Sixty questions, four per chapter. Each answer states the reasoning rather than just the conclusion, and most name the mistake students most often make.

Write your own answer first.

NUU · Fundamentals of Data Science · Student Copy

Chapter 1 — Thinking Like a Data Scientist

Questions

1. Why is “use machine learning to improve education” not yet a good analytical question?
2. Give one example each of a nominal, ordinal, discrete, and continuous variable from the course dataset.
3. What would another analyst need in order to reproduce your work?
4. A colleague reports that “the average student ID is 100,600.” What went wrong, and what does it tell you about their process?

Model answers

1. It names a method and a hoped-for outcome but no population, no measurable variable, no time period, and no decision. “Improve education” is not observable; “improve” against what baseline, measured how? A repaired version: “*Among 2026-cohort students, do weeks 1–4 submissions predict end-of-term withdrawal well enough to support an offer of tutoring?*” That version names who, what is measured, over what period, what claim type (predictive), and what happens next. **Common error:** repairing only the vagueness of “education” while leaving the method fixed. Choosing the method first is the deeper problem — it forces the question to bend toward whatever the method outputs.
2. Nominal: programme — four categories with no order. Ordinal: midterm_satisfaction — ordered 1 to 5, but the gap between 1 and 2 need not equal the gap between 4 and 5, which is why averaging Likert responses is contested. Discrete: submissions — countable, no values between 1 and 2. Continuous: entry_score — any value in a range, limited only by measurement precision. **Common error:** offering student_id as discrete. It is stored as an integer but is nominal; the arithmetic is defined and the meaning is not.
3. The data file *and its version*, since files get corrected; the code; package versions; any random seed; the exact filtering and exclusion rules, including rows dropped and why; manual decisions such as how you recoded a category; and your operational definitions. The practical test is not “did I list these” but “can a colleague open my notebook, restart and run all, and get my numbers.” **Common error:** listing only the code. Code plus a silently updated data file reproduces nothing.
4. They ran describe() and read a row without asking whether the operation was meaningful for that column. The mean of an identifier is arithmetically valid and semantically empty. More importantly, it signals that no one checked what the columns *mean* before summarising them — so any other numeric column that is secretly a code rather than a quantity is also suspect. **Common error:** treating this as a trivial slip. It is a process failure, and the same process will produce a subtler error later.

Chapter 2 — Collecting and Understanding Data

Questions

1. Distinguish target population, sampling frame, and sample using the mid-term survey.
2. Why can a large convenience sample be worse than a small random one?
3. What is a proxy, and what should you do before using one as a model feature?
4. You merge two tables and the row count grows from 1,200 to 1,340. What are the two most likely causes, and how do you distinguish them?

Model answers

1. Target population: all enrolled students, the group the conclusion is about. Sampling frame: students with an active platform account — the list you can actually reach. Sample: the 916 who answered. Each gap loses a systematically different group: students without accounts differ from those with, and non-responders proved to have barely half the engagement of responders. **Common error:** treating the frame and the population as identical because the gap is usually small. The size of the gap is an empirical question, not an assumption.
2. Because the two samples fail in different ways. A random sample's error is *variance* — it shrinks predictably as n grows, and a confidence interval describes it honestly. A convenience sample's error is *bias* — a fixed offset of unknown size and direction that does not shrink at all. Growing a convenience sample tightens the interval around the wrong number, which makes the result more confident and no more correct. **Common error:** saying the convenience sample is "less accurate." It is precisely accurate about a population you are not interested in.
3. A proxy is an observable variable standing in for an unmeasured concept — `minutes_on_platform` for engagement. Before using one: state the concept explicitly; write out at least two concrete scenarios where proxy and concept diverge; check whether those divergences fall unevenly across groups; and confirm the proxy is explainable to whoever is affected by a decision based on it. **Common error:** validating the proxy only against the outcome. A proxy can correlate well with the outcome while being unusable, because its errors concentrate in one group.
4. Either the right-hand table has duplicate keys (so some left rows matched more than once), or the join key is not what you think — for example merging on `student_id` when the right table is keyed on `student_id` and `term`. Distinguish them with `right.duplicated(subset=key).sum()`: if that is non-zero, it is duplicate keys. If it is zero, your key is incomplete. Either way, `validate="many_to_one"` would have raised before the corruption reached your results. **Common error:** fixing it with `drop_duplicates()` after the merge. That hides the cause and may drop legitimate rows.

Chapter 3 — Describing Data: Centre, Spread, and Shape

Questions

1. For a right-skewed income distribution, which is larger, the mean or the median, and why?
2. Why does sample variance divide by $n - 1$?
3. A colleague reports “mean grade 58.8, SD 15.4” for the whole cohort. What is your objection?
4. A student has a z-score of 1.5 on the entry test. What does this tell you about their percentile rank?

Model answers

1. The mean is larger. The mean is a balance point and responds to the *magnitude* of every value, so a long right tail of high incomes drags it upward. The median responds only to *position* — moving the highest income from £200k to £2m does not move the median at all. The size of the gap is itself a rough skew measure. **Common error:** answering “the mean, because it includes outliers.” The median also includes them; it just is not moved by how extreme they are.
2. Because the deviations are measured from the sample mean, not the true population mean. The sample mean is by construction the value minimising the sum of squared deviations for that particular sample, so the deviations are systematically too small as estimates of spread around the population mean. Dividing by $n - 1$ rather than n inflates the estimate by exactly enough to remove that bias on average. The correction is large at small n (12% at $n = 5$) and negligible at large n . **Common error:** “because we lose a degree of freedom.” That is the name of the reason, not the reason. The name is only useful if you can also state the underestimation it corrects.
3. Grades in this dataset are bimodal — three quantitative service courses average 44.1 and the rest average 63.1. A mean of 58.8 falls between the modes and describes neither group; the SD of 13.7 is inflated by the 19-mark gap between groups, when within each group the SD is about 11. Report the two groups separately, with n for each. **Common error:** objecting only that “there might be outliers.” Outliers would be a minor problem; two populations is a different problem, and the fix is disaggregation rather than a robust statistic.
4. By itself, essentially nothing precise. A z-score of 1.5 means the score is 1.5 SDs above the mean, which is a statement about *distance*, not about *rank*. Converting it to a percentile requires assuming a distribution — under a normal distribution it would be roughly the 93rd percentile, but that assumption must be checked, and for a skewed or bimodal variable it can be badly wrong. **Common error:** answering “93rd percentile” without stating the normality assumption. The number is not wrong; the missing conditional is.

Chapter 4 — Data Quality and Cleaning

Questions

1. Give an example of MNAR missingness in the course dataset and explain why no statistical fix exists.
2. Why is `drop_duplicates()` dangerous on records sharing a key but differing in content?
3. A value is 4 standard deviations from the mean. Should you remove it?
4. You coerce a text column to numeric and the row count is unchanged. Is the operation safe?

Model answers

1. `midterm_satisfaction`: students who are dissatisfied are more likely to have disengaged, and disengaged students do not answer surveys. The value's own magnitude drives its absence. No fix exists because correcting the bias requires knowing how the missing responses differ from the observed ones — which is exactly the unobserved quantity. Imputation from observed variables assumes MAR, and if that assumption is wrong it produces a confident, complete, biased dataset. **Common error**: proposing multiple imputation as the solution. Multiple imputation handles MAR well and MNAR not at all; it propagates uncertainty about the *imputation*, not about the *mechanism*.
2. Because it keeps whichever row appears first and drops the rest, with no rule and no warning. If the same `student_id` appears with `entry_score` 64.2 and 71.5, one of those is the current value and the function has no way to know which — so the answer depends on file ordering, which depends on how the export was sorted. The fix is to make the rule explicit: sort by a timestamp, then `keep="last"`, and record in the cleaning log that later records supersede earlier ones. **Common error**: treating it as a rare edge case. In our small export it affects three students; in a production system fed by multiple sources it is routine.
3. Not on that basis alone. Four SDs tells you the value is unusual relative to a mean and SD that the value itself helped compute. The question is whether it is *impossible* — outside the definitional range of the variable — or merely rare. An `age_at_entry` of 220 is impossible and should be set missing. An `age_at_entry` of 47 is four SDs out and is a real mature student, and removing it deletes part of the population. Keep it, flag it, and report a sensitivity analysis with and without. **Common error**: applying a 3-SD rule as policy. In this dataset it would remove almost exclusively mature part-time students, converting a cleaning step into an undisclosed change of population.
4. No. Row count is the wrong check, because `errors="coerce"` preserves every row and replaces unparseable values with NaN. The operation could have destroyed a thousand values without changing the shape by one. The correct check is to compare the missing count before and after: `after.isna().sum() - before.isna().sum()` should equal zero, or equal a number you can explain. **Common error**: checking dtype instead. The dtype will be `float64` whether one value converted or all of them did.

Chapter 5 — Transformation, Features, and Leakage

Questions

1. Why does a decision tree not require feature scaling, while k-nearest-neighbours does?
2. What is the dummy variable trap, and which models does it affect?
3. Give an example of target leakage from the course dataset.
4. You fit `SimpleImputer` on the full dataset, then split. What exactly has leaked, and would it show up as an error?

Model answers

1. A tree splits on thresholds within a single feature at a time — “is `entry_score` above 62?” — and any monotone rescaling maps that threshold to a corresponding one, leaving the tree structure identical. k-NN computes distance across all features jointly, so a feature’s contribution is proportional to its numeric range. With `entry_score` spanning about 60 units and `age_at_entry` about 30, unscaled distance is dominated by score, and the neighbourhoods are effectively determined by one variable. **Common error:** saying trees “don’t care about units.” They do not care about *scale*, but they are affected by feature *interactions* the scaling might have been intended to expose.
2. With k categories one-hot encoded into k columns, the columns sum to 1 in every row, making them perfectly collinear with the intercept. The design matrix is singular, so coefficients have no unique solution. Dropping one level fixes it: that level is absorbed into the intercept and becomes the reference against which the others are read. It affects ordinary least squares and any model requiring matrix inversion. Regularised models (ridge, lasso) and trees are unaffected, because regularisation makes the solution unique and trees do not invert anything. **Common error:** always dropping a level. With ridge or a random forest, keeping all k is fine and sometimes more interpretable.
3. Predicting `withdrew` from total activity across all fourteen weeks. A student who withdrew in week 7 has no activity in weeks 8–14 *because* of the outcome, so the feature is partly a copy of the target. It lifts test ROC-AUC from 0.701 to 0.775 and would collapse in deployment, because at the week-5 decision point the feature does not exist. **Common error:** offering `passed` as a predictor of `final_grade`. That is leakage but a trivial kind — it is a deterministic function of the target and anyone would catch it. The dangerous cases are the plausible ones.
4. The median (or mean) used to fill missing values was computed using test-set rows, so every training row was imputed with a value that depends on data the model is later evaluated on. The test score is therefore optimistic. **It produces no error, no warning, and no visible symptom** — the code runs, the shapes are right, and the number is simply too high by an unknown amount. The only defence is structural: put the imputer in a `Pipeline` so it is refitted on training folds only. **Common error:** assuming the effect is negligible because a median is “robust”. Robustness to outliers is unrelated to whether the statistic saw the test set.

Chapter 6 — Exploratory Data Analysis

Questions

1. What four properties should you inspect in a numerical distribution?
2. How can a missingness pattern create an apparent relationship?
3. Explain Simpson's paradox in your own words, using the study-mode example.
4. A correlation of 0.68 survives stratification by entry score. What does that establish, and what does it not?

Model answers

1. Modality (one peak or several — several usually means mixed populations), skew (which tail is longer, and hence whether mean and median will disagree), spread (in interpretable units, with a robust measure alongside the SD if the tail is heavy), and outliers (present, and impossible or merely unusual). Modality comes first because a bimodal distribution invalidates almost every single-number summary you might compute next. **Common error:** listing “normality” as one of them. Whether a variable is normal matters for some inferential procedures and not for description; modality and skew are the descriptive facts.
2. If the probability of being missing depends on the value of another variable, dropping incomplete rows changes the composition of the sample in a way that is correlated with that variable. In our data, `midterm_satisfaction` is missing for 51.4% of the least-engaged quintile and 4.3% of the most engaged. Any analysis of satisfaction on complete cases is therefore conducted on a sample skewed toward engaged students — so a relationship between satisfaction and any engagement-correlated variable will be attenuated or distorted, and one may appear that does not exist in the full population. **Common error:** assuming a high overall completion rate protects you. 76% complete is fine if the 24% are random and useless if they are not.
3. An association can point one way in the pooled data and the opposite way inside every subgroup, because the pooled comparison also compares subgroup composition. Part-time students pass 77.0% of registrations against 74.5% for full-time. But 25% of full-time registrations are in the three low-pass-rate quantitative courses against only 6.8% of part-time ones — part-time students defer those courses. So the aggregate mostly measures *which courses each group sat*, and within each course part-time students actually pass less often, in eleven of twelve cases. **Common error:** describing it as a statistical anomaly or a paradox in the data. Both numbers are correct; the paradox is in the inference, and the resolution is always a composition difference you can point to.
4. It establishes that entry score does not account for the association — the relationship is not simply prior preparation appearing in two places. That is real evidence and it rules out one specific rival explanation. It does not establish that logging in causes better grades. Reverse causation remains open (students who understand the material return to the platform), as does confounding by any variable you did not stratify on — motivation, available study time, employment. Stratification addresses the confounders you named, and observational data

cannot rule out the ones you did not. **Common error:** treating survival under stratification as approaching causal evidence. It raises the observation's credibility as an *association*; the rung has not changed.

NUU · Fundamentals of Data Science · Student Copy

Chapter 7 — Visualization and Data Storytelling

Questions

1. Why must bar charts start at zero when line charts often need not?
2. Name two problems with encoding twelve categories by colour.
3. What is a message title, and when is one inappropriate?
4. A chart shows means with error bars and no caption. Why can you not interpret it?

Model answers

1. Because the two forms encode different things. A bar's *length* encodes the value, so length is proportional to magnitude only if the baseline is zero — truncate it and the bar's length encodes an arbitrary quantity. A line encodes *change* through its slope and shape, and slope is unaffected by where the axis starts, so a non-zero baseline can legitimately reveal variation that a zero baseline would flatten. The obligation with a line chart is to label the range clearly so nobody reads magnitude off it. **Common error:** stating the rule as “never truncate an axis.” That over-corrects and would make many time-series charts useless.
2. First, perceptual: readers cannot hold twelve colour-to-category mappings in working memory, so the chart requires constant reference back to the legend and no comparison is actually made. Second, accessibility: twelve colours from a default palette will contain pairs that are indistinguishable under red–green colour-vision deficiency, which affects around 8% of men — for those readers two categories silently merge. A third, deeper problem: needing twelve colours usually signals that the question was never narrowed, and the fix is faceting or highlighting rather than a better palette. **Common error:** proposing a colour-blind-safe twelve-colour palette. No such palette is perceptually distinguishable at twelve levels; the encoding is wrong, not the colours.
3. A title stating the chart's finding rather than its contents — “Submissions fall 22% after week 8” instead of “Submissions by week.” It is more useful because the reader gets the point immediately, and more honest because a claim can be challenged while a label cannot. It is inappropriate when the claim it states sits on a higher rung than the evidence supports: “Students disengage after week 8” is an interpretation resting on a proxy, and “The new timetable caused submissions to fall” is causal and unsupported by a line chart. It is also inappropriate for genuinely exploratory figures where you are showing a reader the data rather than asserting a conclusion. **Common error:** treating message titles as always better. A message title on weak evidence launders an unsupported claim through the authority of a figure.
4. Because error bars can represent at least three different quantities — the standard deviation of the data, the standard error of the mean, or a 95% confidence interval — and these differ by large factors. For $n = 100$, the SE is one tenth the SD, and a 95% CI is roughly twice the SE. So identical-looking bars could indicate spread in the population, precision of the estimate, or a plausible range for the true mean, and without a label you cannot tell which. Whether two

groups' bars overlap therefore means nothing until you know what they are. **Common error:** assuming a convention. There is none, and even within one field practice varies.

NUU · Fundamentals of Data Science · Student Copy

Chapter 8 — Probability and Bayesian Reasoning

Questions

1. A test is 99% accurate for a disease affecting 1 in 10,000. You test positive. Roughly what is the probability you have it?
2. Why is ?
3. When does ?
4. Your simulation of 10 million trials gives 0.2387. How confident should you be in that number?

Model answers

1. About 1%. Take 1,000,000 people: 100 have the disease, 999,900 do not. At 99% accuracy the test finds 99 of the 100 true cases and falsely flags 1% of the healthy — about 9,999 people. Total positives $\approx 10,098$, of which 99 are real, giving $\frac{99}{10,098}$. Even a 99%-accurate test is wrong about 99 times in 100 when the condition is this rare. **Common error:** answering 99%. That is, the test's property, not, which is what you asked about.
2. Because they condition on different things and are scaled by different base rates. and — the same numerator over different denominators. They are equal only when. Concretely: nearly all students who withdraw had low engagement, but only a minority of students with low engagement withdraw, because low engagement is far more common than withdrawal. **Common error:** explaining it as “correlation is symmetric but causation isn't.” Correlation is indeed symmetric; conditional probability is not, and that is a separate arithmetic fact independent of causation.
3. Only when A and B are independent — that is, when, so knowing B occurred tells you nothing about A. The general rule is always; the product form is a special case. Independence is an assumption about the world, not a mathematical default, and it fails whenever a shared cause is present. **Common error:** confusing independent with mutually exclusive. Mutually exclusive events are maximally *dependent* — if one occurs the other cannot — and satisfy, not.
4. Confident about its *precision*, not its *accuracy*. With 10 million trials the Monte Carlo standard error is about, so the simulation has converged tightly on the value its model implies — the fourth decimal is stable. But that value is only correct if the model is. A wrong assumption (independence where correlation exists, a misspecified rate) produces a precisely-estimated wrong answer, and adding trials narrows the interval around it without moving it toward the truth. The check for accuracy is varying the model and comparing to an analytical result, not increasing n. **Common error:** reporting all four decimals as though precision were evidence of correctness.

Chapter 9 — Statistical Inference and Experiments

Questions

1. Interpret a 95% confidence interval of $[-0.89, 1.74]$ for a difference in means.
2. Why can a tiny effect be highly significant?
3. What does a p-value of 0.03 mean, precisely?
4. A team runs 20 comparisons and reports the two with $p < 0.05$. What is wrong?

Model answers

1. The interval was produced by a procedure that captures the true difference in 95% of repeated samples. Substantively, it is consistent with no difference (it contains zero) and — more informatively — it rules out any difference larger than about 1.8 marks in either direction. Since the smallest difference the university would act on is far larger than that, this is a positive finding of *practical equivalence*, not merely a failure to find something. **Common error:** saying “there is a 95% probability the true difference lies in this interval.” Under the frequentist framework the true difference is fixed and the interval is random; the probability statement attaches to the procedure across repetitions, not to this interval.
2. Because the p-value depends on the effect size *and* the standard error, and the standard error shrinks as n increases. A difference of 0.3 minutes with SD 12 gives $p = 0.86$ at $n = 100$ and $p = 0.0004$ at $n = 40,000$ — the effect is identical, and only the precision changed. With sufficient data, any difference that is not exactly zero becomes statistically significant, so “significant” answers “can we distinguish this from zero”, not “does it matter”. **Common error:** concluding that large samples are therefore misleading. They are not; they are precise. The error is in treating significance as a decision rule instead of stating a minimum effect of interest in advance.
3. If the null hypothesis were true, and the model’s assumptions held, data at least this extreme would occur in 3% of repeated samples. It is *not* the probability that the null is true, not the probability the result is due to chance, not the probability the finding will replicate, and not a measure of effect size. It also says nothing about the alternative hypothesis — the data could be improbable under the null and equally improbable under the alternative. **Common error:** “there is a 3% chance the result is a fluke.” That reverses the conditional in exactly the way Chapter 8 warns about.
4. Running 20 independent tests at $\alpha = 0.05$ gives a 64% chance that at least one comes back significant even when nothing is going on, so two hits out of 20 is close to what pure noise produces. Reporting only the two that cleared the threshold, and suppressing the eighteen that did not, makes the evidence look far stronger than it is. The fixes: pre-register a single primary outcome; apply a correction (Bonferroni would require $p < 0.0025$ here, Benjamini–Hochberg is less conservative); and report all 20 comparisons regardless of outcome. **Common error:** treating the correction as sufficient on its own. A correction handles the arithmetic; it does not undo the selective reporting, and only disclosing all tests does that.

Chapter 10 — Regression and the Machine-Learning Workflow

Questions

1. Why fit a baseline before any real model?
2. Interpret the coefficient 0.517 on `entry_score`.
3. RMSE is much larger than MAE. What does that tell you?
4. Your model gets $R^2 = 0.95$ on training data and 0.31 on test data. Diagnose it.

Model answers

1. Because a model's score has no meaning in isolation. MAE 7.98 could be excellent or worthless depending on whether predicting the mean gives 8.1 or 25. The baseline converts an absolute number into a claim about value added — here, a 10.1% reduction in mean absolute error. It also catches implementation bugs: a model that fails to beat the mean usually has a broken feature matrix or target rather than a subtle modelling problem. **Common error:** using R^2 as an implicit baseline. R^2 does compare against the mean, but it is scaled in variance units and does not tell you whether the improvement matters in the units of the decision.
2. Comparing two students whose entry scores differ by one mark, the model predicts their DS100 final grades will differ by 0.517 marks. In a model with other predictors, add: “among students with the same values of those other predictors.” It does not mean that increasing a given student's entry score would raise their final grade — that is a causal claim about an intervention, and these are observational data where entry score is correlated with preparation, motivation, and circumstance. **Common error:** the phrasing “a one-unit increase in x leads to a 0.517 increase in y ”. “Increase” and “leads to” both describe an intervention; “differ by” describes a comparison.
3. That a small number of large errors dominate the error distribution, because squaring gives them disproportionate weight. In the example, one residual of 20 among five raises RMSE 68% above MAE; removing it brings the two within 3% of each other. The practical response is to find those residuals and diagnose them — they may be data errors, a subgroup the model handles badly, or a genuinely unpredictable tail. **Common error:** treating the gap as a defect to be minimised. It is information. A model with $RMSE \approx MAE$ has uniform errors; whether that is better depends entirely on the cost function.
4. Overfitting. The model has learned patterns specific to the training sample — noise, not signal — which is why performance collapses on data it has not seen. The gap of 0.64 is large enough that the training score carries essentially no information about deployment behaviour. Remedies: reduce model complexity, add regularisation (ridge or lasso), gather more training data, or reduce the feature count. Verify with a learning curve — if training and validation scores converge as n grows, more data will help; if they plateau apart, the model is too flexible. **Common error:** reporting the training R^2 alongside the test R^2 as though both describe the model. Only the test figure estimates generalisation; the training figure describes memorisation.

Chapter 11 — Classification I: Building a Classifier

Questions

1. Why not use linear regression for a binary outcome?
2. A coefficient is -0.422 on a standardised feature. Interpret it.
3. Your classifier gets 88.6% accuracy. Is it good?
4. Why does k-NN require scaling when a decision tree does not?

Model answers

1. Two reasons. Predictions are unbounded, so the model returns values like -0.3 and 1.4 , which cannot be probabilities and cannot be thresholded sensibly. And the error structure is wrong: for a binary outcome the residuals take only two values at each fitted point and their variance depends on the fitted value, violating the assumptions behind every standard error, interval, and p-value the regression reports. Logistic regression fixes both by modelling the log-odds and fitting by maximum likelihood. **Common error:** saying linear regression “would not work”. It runs and produces a fit; the problem is that the output is uninterpretable and the inference is invalid, which is worse than failing loudly.
2. A one-standard-deviation increase in that feature is associated with a change of -0.422 in the log-odds of the outcome — equivalently, it multiplies the odds by $e^{-0.422}$, a 34% reduction in odds, holding the other features constant. Because the feature was standardised, this is comparable in magnitude with the other coefficients. It is **not** a 34% reduction in probability: at a base probability of 0.115 the odds are 0.130, multiplying gives 0.085, and converting back gives $0.085 / 0.130 \approx 0.65$ — a fall of 3.6 percentage points, not 34. **Common error:** reading the odds ratio as a risk ratio. They coincide only when the outcome is very rare, and diverge sharply otherwise.
3. No, and the number is actively misleading. 88.5% of students do not withdraw, so a model predicting “nobody withdraws” achieves 88.6% accuracy while identifying zero at-risk students — the exact opposite of what the model exists to do. Accuracy is dominated by the majority class whenever classes are imbalanced. Report ROC-AUC (which is 0.500 for the trivial model), PR-AUC against the 0.115 baseline, and recall at the operating threshold you intend to use. **Common error:** proposing to fix this by balancing the classes. Resampling changes the training distribution and can help the fit, but it does not make accuracy an appropriate metric for a rare outcome.
4. k-NN’s entire model is the distance function, and Euclidean distance sums squared differences across features in their raw units. A feature spanning 0–100 contributes vastly more than one spanning 17–47, so the neighbourhoods — and therefore every prediction — are determined by whichever feature happens to have the widest numeric range. A tree evaluates one feature at a time against a threshold it chooses from that feature’s own values, so rescaling maps every candidate split to a corresponding one and the tree structure is unchanged. **Common error:** concluding trees are therefore preferable. Scale-invariance is convenient, not a quality advantage; trees have their own weaknesses, notably instability and poor calibration.

Chapter 12 — Classification II: Errors, Thresholds, and Fairness

Questions

1. Your model has 88.9% accuracy and 2.4% recall on a rare outcome. Explain how both are true.
2. When should you prefer a precision–recall curve to an ROC curve?
3. How should the threshold be chosen?
4. A model has equal accuracy across two groups. Is it fair?

Model answers

1. 88.6% of students do not withdraw, so a model that almost never predicts “withdraws” is right about nearly every negative case and accumulates high accuracy from the majority class alone. At threshold 0.50 this model flagged one student out of 360 and found 1 of the 41 who withdrew — hence recall of $1/41 = 2.4\%$. Accuracy counts all four cells of the confusion matrix and is therefore dominated by the 319 true negatives; recall counts only the positive row, which is the row the model exists to get right. **Common error:** describing this as the model being “broken”. It is functioning exactly as trained — log loss on imbalanced data with a 0.5 cut produces this. The fault is in the metric and the threshold, not the fit.
2. When positives are rare and the cost of false positives is borne by real people. ROC’s x-axis is the false positive rate, whose denominator is all negatives — with 319 negatives, adding 96 false positives moves FPR by 0.30 but the curve still looks respectable. Precision’s denominator is the number flagged, so those same 96 false positives crush precision from 1.00 to 0.20. PR also has an honest baseline: the positive rate, 0.115 here, against which a PR-AUC of 0.236 reads as roughly a doubling rather than as a poor score. Use ROC to compare models on ranking ability; use PR to decide whether the model is usable. **Common error:** treating them as interchangeable summaries. They answer different questions and can rank two models differently.
3. From the consequences, not from the data. Establish what happens after a positive prediction, what a false positive costs and to whom, what a false negative costs and to whom, and how many flags the organisation can actually act on. Here a false negative is a student who needed help and was not offered it, and a false positive is an unnecessary email — asymmetric costs favouring recall, bounded by adviser capacity. If three advisers can each contact 40 students, the threshold is whichever value flags about 120, and the resulting recall is what the policy buys. **Common error:** picking the threshold that maximises F1 or Youden’s J. Those optimise a mathematical criterion that encodes an error-cost ratio nobody chose, and usually a wrong one.
4. No — equal accuracy is close to no evidence at all. Accuracy is dominated by the majority class, so two groups can post identical accuracy while one has 80% recall and the other 45%: the model finds at-risk students in one group and misses them in the other, and the aggregate hides it entirely. Audit recall, precision, and flag rate separately per group, alongside each group’s base rate and n. Note also that the three standard fairness criteria are mutually

incompatible when base rates differ, so “fair” is not a property to verify but a criterion to choose and defend against what the flag does. **Common error:** treating the audit as the whole obligation. A model can pass every parity check and still be indefensible if the people it affects cannot find out they were flagged or contest the decision.

NUU · Fundamentals of Data Science · Student Copy

Chapter 13 — Clustering and Dimensionality Reduction

Questions

1. Why must features be scaled before k-means?
2. The elbow plot shows no clear bend. What do you do?
3. What does the first principal component represent?
4. Your silhouette score is 0.32. Is that good?

Model answers

1. Because k-means minimises Euclidean distance, and distance sums squared differences in raw units — so a feature with a wider numeric range contributes proportionally more to every distance and effectively decides the partition. With `entry_score` spanning 64 units and `age_at_entry` spanning 30, score contributes about 4.5 times as much variance, and the clusters come out as entry-score bands. Scaling asserts that one SD of each feature matters equally, which is itself an assumption — but a stated one, rather than an accident of the units the registry recorded. **Common error:** describing scaling as “improving accuracy”. Clustering has no accuracy; scaling changes which grouping you get, and neither is correct in the abstract.
2. Accept that the data may not contain discrete clusters and stop looking for a number to discover. Run a permutation null test — shuffle each column independently and recompute the silhouette — to check whether the apparent structure exceeds what k-means produces on structureless data of the same shape. Check stability across random seeds with the adjusted Rand index. Then choose k from what you will do with it: three clusters because student services can run three interventions is a defensible justification, and a curve that fails to disagree is corroboration rather than evidence. **Common error:** switching to a different criterion until one shows a peak. Trying elbow, then silhouette, then gap statistic until something looks decisive is metric-shopping.
3. The direction in feature space along which the data vary most — the linear combination of features with maximum variance, subject to unit length. Its loadings say how much each original feature contributes. On our data PC1 explains 46.8% of total variance and loads heavily on the engagement features, so it is reasonable to *call* it an engagement axis. It is not a cause, not a latent trait, and not something the data established exists; it is an arithmetic property of the covariance matrix, and its interpretation is yours and should be held loosely. **Common error:** treating a high explained-variance ratio as validation of the interpretation. It validates that the direction captures variation, not that the name is right.
4. Weak. Silhouette runs from -1 to 1; values above about 0.5 suggest reasonably separated clusters, 0.25 to 0.5 suggests substantial overlap, and below 0.25 suggests little structure. At 0.32 the clusters overlap considerably, and the honest report says so rather than presenting three tidy segments. The more useful question is whether 0.32 exceeds what the same algorithm produces on permuted data — if a null run also gives around 0.30, the score is evidence of nothing at all. **Common error:** comparing the silhouette across different k values only. That identifies the best k *given* that you are clustering; it never tests whether you should be.

Chapter 14 — Time Series and Text

Questions

1. Why is k-fold cross-validation invalid for time series?
2. What does the seasonal naive baseline predict, and why is it strong here?
3. Why does `rolling(7)` without `shift(1)` leak?
4. Your sentiment classifier is 89.5% accurate. What can you conclude about the 10.5%?

Model answers

1. Because it trains on data from after the test period. Standard k-fold assigns rows to folds at random, so a fold can contain 20 March in training and 19 March in test — and since consecutive days are strongly autocorrelated, the model is effectively shown the answer. The resulting score measures interpolation between known points, not forecasting, and will be far better than deployment performance. Use a chronological split or `TimeSeriesSplit`, which trains on a prefix and tests on the block immediately after. **Common error:** thinking shuffling is the problem. Shuffling is one symptom; the underlying issue is that the rows are not exchangeable, which also invalidates independence-based standard errors even without any splitting.
2. It predicts that a given day equals the same weekday one week earlier. It is strong here because weekly seasonality is the dominant structure in the series — weekday means range from 491 sessions on Sunday to 1,325 on Thursday, a 2.7-fold swing — so copying last week's matching day automatically captures the largest source of variation. Its MAE of 88.9 beats the naive baseline's 345.8 and the mean's 338.0 by a factor of nearly four. Any model that fails to beat 88.9 has added nothing over looking up last Tuesday. **Common error:** comparing only against the naive or mean baseline. Those are trivially weak on seasonal data, and beating them proves nothing.
3. Because `rolling(7)` computes a window ending at the current row, so the mean includes the value you are trying to predict. The feature for day t contains sessions from day t , which will not be known at prediction time. `.shift(1)` moves the series back one step before the window is applied, so the mean covers days $t-7$ through $t-1$ only. This produces no error, no warning, and no visible symptom — just an optimistic score. **Common error:** using `center=True`, which is worse: it puts half the window in the future.
4. That it is concentrated, not random — and that it sits in the most informative comments. Reading the misclassified cases shows two patterns: negation (“not clear at all... but it became well structured”) where bag-of-words cannot attach *not* to *clear*, especially once *not* is removed as a stop word; and mixed-verdict comments (“really engaging lecturer, the course was poorly organised”) where the model averages competing tokens with no way to identify which clause carries the overall judgement. These are limits of the representation, not of the fit, so more data or more tuning will not remove them. Bigrams help with the first pattern (raising accuracy to 92.3%) and not the second. **Common error:** reporting 89.5% as though the errors were evenly

distributed noise. For aggregate counting the model is fine; for surfacing individual comments to a course leader it fails precisely on the comments worth surfacing.

NUU · Fundamentals of Data Science · Student Copy

Chapter 15 — Dashboards, Delivery, and Responsible Practice

Questions

1. Why is suppressing one small cell in a row insufficient?
2. What makes a dashboard a decision product rather than a report?
3. Why write limitations before conclusions?
4. Your model is deployed and withdrawals fall. Did it work?

Model answers

1. Because if the row's margin is published, the suppressed value can be recovered by subtraction. For ST110: the published overall pass rate (0.214, $n = 229$) and full-time rate (0.218, $n = 220$) give part-time passes as $0.214 \times 229 - 0.218 \times 220 = 1$, and with $n = 9$ the suppressed rate is $1/9 = 0.111$. The fix is complementary suppression — hide a second cell in the row so no unique solution exists — or suppress the margin. Differencing also works across time, so suppression rules must stay consistent between releases. **Common error:** thinking rounding solves it. A rounded cell plus a published total still bounds the hidden value tightly, often to a single possibility.
2. That every state it can display has an action attached to it. A decision product answers “who uses this, how often, to decide what, and what do they do differently depending on what they see.” If no view changes anyone's behaviour, it is a report that someone has to maintain — with all the staleness and breakage risk of live software and none of the benefit. The design questions therefore come before any code: who, how often, what action, what must never be shown, and how a user learns the data are stale. **Common error:** judging a dashboard by how much it displays. Comprehensiveness is usually a symptom of nobody having decided what it is for.
3. Because writing them last means writing them against a conclusion you are already committed to, and they get softened to fit. Written first, they constrain what the conclusion is allowed to claim — and if the limitations make the intended conclusion untenable, you have discovered that while it is still cheap to fix. It also improves the limitations themselves: drafted early they are an honest inventory, drafted late they become a defensive list of things the reader might otherwise object to. **Common error:** treating limitations as a disclaimer protecting the author. Their function is to tell the reader what the result can bear, which is what makes the rest of the report usable.
4. You cannot tell. Withdrawals could have fallen because of the intervention, or because this cohort differed, because the assessment schedule changed, because of regression to the mean after an unusually bad prior year, or because of anything else that changed at the same time. Comparing flagged-and-contacted students with unflagged ones is worse than useless, since the two groups differ in exactly the risk factors the model selected on. Establishing that it worked requires a randomised control group — students contacted regardless of score, or flagged students randomly assigned to contact or not — specified before deployment. **Common error:**

comparing this year's rate with last year's. That is a before-and-after comparison with no control, and it will confirm whatever you hoped for roughly half the time.

NUU · Fundamentals of Data Science · Student Copy

Part 2 — Code cookbook

Organised by what you are trying to do, not by chapter. Every snippet runs against the course dataset.

Assume this at the top of any notebook:

```
from google.colab import drive
drive.mount('/content/drive')
DATA = "/content/drive/MyDrive/fds/data/"
```

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
```

```
students = pd.read_csv(DATA + "students.csv")
enrolments = pd.read_csv(DATA + "enrolments.csv")
activity = pd.read_csv(DATA + "activity.csv")
outcomes = pd.read_csv(DATA + "outcomes.csv")
```

Loading and first inspection

```
# Identifiers as text, dates parsed. Do this on every load.
df = pd.read_csv(DATA + "students.csv",
                 dtype={"student_id": str},
                 parse_dates=["registered_on"])
```

```
df.shape                # rows, columns
df.dtypes               # what pandas thinks each column is
df.head(3)              # actual rows, not a summary
df.describe().round(2)  # numeric summary
df.isna().sum()         # missing per column
df["programme"].value_counts(dropna=False)
```

```
# The identifier really identifies
assert df.student_id.is_unique
```

Selecting and filtering

```
df.loc[df.entry_score > 60, ["student_id", "entry_score"]]
df.query("entry_score > 60 and programme == 'Biology'")
df[df.programme.isin(["Biology", "Economics"])]
df[df.entry_score.between(50, 70)]
df[df.midterm_satisfaction.isna()]
```

Grouping and aggregating

```
# Named aggregations – clearer than passing a dict
enrolments.groupby("course_code").agg(
    n=("student_id", "nunique"),
    mean_grade=("final_grade", "mean"),
    sd=("final_grade", "std"),
).round(2)
```

```
# Two grouping variables, laid out as a table
```

```

enrolments.merge(students, on="student_id").pivot_table(
    index="course_code", columns="study_mode",
    values="passed", aggfunc="mean").round(3)

# Proportion within each group
pd.crosstab(df.programme, df.study_mode, normalize="index").round(3)

```

Merging without breaking anything

```

before = len(enrolments)
merged = enrolments.merge(students, on="student_id",
                           how="left",
                           validate="many_to_one") # raises on
duplicate keys
assert len(merged) == before

# Check nothing was dropped
print("unmatched:",
      (~enrolments.student_id.isin(students.student_id)).sum())

```

Cleaning

```

# Whitespace and case in a categorical
df["programme"] = df.programme.str.strip().str.title()

# Text that should be numeric
df["entry_score"] = pd.to_numeric(
    df.entry_score.astype(str).str.rstrip("%"), errors="coerce")
# ALWAYS check what coerce destroyed
print("new missing:", df.entry_score.isna().sum() -
      known_missing_before)

# Impossible values become missing, not guesses
bad = (df.age_at_entry < 16) | (df.age_at_entry > 70)
df.loc[bad, "age_at_entry"] = pd.NA

# Mixed date formats
df["registered_on"] = pd.to_datetime(df.registered_on,
                                     format="mixed", dayfirst=True)

# Duplicates: state the rule, do not let the file order decide
clean = (df.sort_values("registered_on")
         .drop_duplicates(subset="student_id", keep="last"))

# Guardrails that fail loudly next time
assert clean.student_id.is_unique
assert clean.entry_score.dropna().between(0, 100).all()

```

Transformation and pipelines

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

```

```

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

numeric      = ["early_logins", "early_submissions", "entry_score"]
categorical   = ["programme", "study_mode"]

pre = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore",
                          sparse_output=False), categorical),
])

model = Pipeline([("pre", pre),
                  ("clf", LogisticRegression(max_iter=1000))])

```

Everything inside a Pipeline is refitted per fold. That is the point — it makes leakage structurally difficult rather than merely discouraged.

Plotting

```

OKABE_IT0 = ["#0072B2", "#E69F00", "#009E73", "#D55E00", "#CC79A7",
             "#56B4E9"]

plt.rcParams.update({
    "figure.dpi": 150, "font.size": 10.5,
    "axes.titlesize": 12, "axes.titleweight": "bold",
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.alpha": 0.25,
})

fig, ax = plt.subplots(figsize=(7, 3.6))
ax.bar(means.index, means.values, color=OKABE_IT0[0], width=0.6)
ax.set_ylim(0, 100) # zero baseline on every bar chart
ax.set_ylabel("mean final grade")
ax.set_title("Mean grades differ by under 2 marks across programmes")

# Small multiples beat twelve colours
fig, axes = plt.subplots(3, 4, figsize=(12, 7), sharex=True,
sharey=True)
for ax, (name, g) in zip(axes.flat, df.groupby("course_code")):
    ax.plot(g.iso_week, g.logins, color=OKABE_IT0[0])
    ax.set_title(name, fontsize=9)

# The greyscale test
plt.savefig("chart.png", dpi=150, bbox_inches="tight")
from PIL import Image
Image.open("chart.png").convert("L").save("chart_grey.png")

```

Splitting and evaluating

```
from sklearn.model_selection import train_test_split, cross_val_score
```

```
# stratify when a class is rare
```

```
X_tr, X_te, y_tr, y_te = train_test_split(  
    X, y, test_size=0.3, random_state=0, stratify=y)
```

```
scores = cross_val_score(model, X, y, cv=5, scoring="roc_auc")
```

```
print(f"{scores.mean():.3f} ( $\pm$ {scores.std():.3f})") # always report  
the spread
```

Scoring strings you will use: "neg_mean_absolute_error",
"neg_root_mean_squared_error", "r2", "accuracy", "roc_auc",
"average_precision", "f1".

Baselines

```
from sklearn.dummy import DummyRegressor, DummyClassifier
```

```
DummyRegressor(strategy="mean") # predicts the training  
mean
```

```
DummyClassifier(strategy="most_frequent") # predicts the majority  
class
```

```
DummyClassifier(strategy="stratified") # predicts at the base  
rate
```

A score without a baseline is uninterpretable. Fit one first, every time.

Classification metrics

```
from sklearn.metrics import (confusion_matrix, precision_score,  
recall_score,
```

```
                             f1_score, roc_auc_score,
```

```
average_precision_score,
```

```
                             classification_report)
```

```
p = model.predict_proba(X_te)[: , 1]
```

```
tn, fp, fn, tp = confusion_matrix(y_te, (p >= 0.15)).ravel()
```

```
print(classification_report(y_te, (p >= 0.15)))
```

```
print("ROC-AUC:", roc_auc_score(y_te, p))
```

```
print("PR-AUC: ", average_precision_score(y_te, p)) # use when  
positives are rare
```

```
# Threshold sweep – this is how you choose an operating point
```

```
for t in np.arange(0.05, 0.65, 0.05):
```

```
    pred = (p >= t).astype(int)
```

```
    print(f"t={t:.2f} flagged={pred.sum():4d} "
```

```
          f"prec={precision_score(y_te, pred, zero_division=0):.3f} "
```

```
          f"rec={recall_score(y_te, pred):.3f}")
```

Clustering and PCA

```
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, adjusted_rand_score

Z = StandardScaler().fit_transform(df[cols])          # scaling is not
optional here
km = KMeans(n_clusters=3, n_init=20, random_state=0).fit(Z)
print(silhouette_score(Z, km.labels_))

# Is the structure real? Shuffle each column and cluster the result.
rng = np.random.default_rng(0)
Z_null = np.column_stack([rng.permutation(Z[:, j]) for j in
range(Z.shape[1])])
km_null = KMeans(3, n_init=10, random_state=0).fit(Z_null)
print("null silhouette:", silhouette_score(Z_null, km_null.labels_))

# Stability across seeds
labs = [KMeans(3, n_init=10, random_state=s).fit_predict(Z) for s in
range(10)]
print([round(adjusted_rand_score(labs[0], l), 3) for l in labs[1:]])

pca = PCA().fit(Z)
print(pca.explained_variance_ratio_.round(3))
```

Time series

```
daily = pd.read_csv(DATA + "platform_daily.csv",
                    parse_dates=["date"]).sort_values("date")

daily["lag_1"] = daily.sessions.shift(1)
daily["lag_7"] = daily.sessions.shift(7)
daily["roll_7"] = daily.sessions.shift(1).rolling(7).mean()  # shift
BEFORE roll
daily["dow"] = daily.date.dt.dayofweek

# Chronological split. Never random.
cut = int(len(daily) * 0.8)
train, test = daily[:cut], daily[cut:]

from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5)

# The baseline to beat
snaive = daily.sessions.shift(7)[cut:]
print("seasonal naive MAE:", np.abs(test.sessions -
snaive).mean().round(1))
```

Text

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import cross_val_predict

pipe = make_pipeline(TfidfVectorizer(stop_words="english",
                                     ngram_range=(1, 2)),
                    LogisticRegression(max_iter=1000))

# Read the mistakes – they are never random
pred = cross_val_predict(pipe, feedback.comment, feedback.sentiment,
                        cv=5)
wrong = feedback[pred != feedback.sentiment]

# What did it learn?
pipe.fit(feedback.comment, feedback.sentiment)
vec, lr = pipe.named_steps["tfidfvectorizer"],
pipe.named_steps["logisticregression"]
coefs = pd.Series(lr.coef_[0],
                 index=vec.get_feature_names_out()).sort_values()
print(coefs.head(8), coefs.tail(8))
```

Simulation

```
rng = np.random.default_rng(42) # always seed

rng.random(n) # uniform [0,1)
rng.normal(mu, sigma, n)
rng.integers(low, high, size)
rng.choice(population, size, replace=False)
rng.permutation(array)

# Bootstrap a confidence interval
boot = [rng.choice(x, len(x), replace=True).mean() for _ in
        range(10_000)]
print(np.percentile(boot, [2.5, 97.5]).round(2))

# Permutation test
observed = A.mean() - B.mean()
combined = np.concatenate([A, B])
null = [(lambda s: s[:len(A)].mean() - s[len(A):].mean())
        (rng.permutation(combined))
        for _ in range(10_000)]
print("p =", (np.abs(null) >= abs(observed)).mean())
```

Part 3 — Formula reference

Description

Quantity	Formula	Note
Sample mean		Uses every value; pulled by extremes
Sample variance		because deviations are from
Standard deviation		Same units as the data
IQR		Robust; ignores the tails
z-score		Distance in SD units, not a percentile

Inference

Quantity	Formula	Note
Standard error of mean		Quadruple to halve
Confidence interval		Describes the procedure, not this interval
SE of a difference		Welch; does not assume equal variance
Cohen's d		0.2 small, 0.5 medium, 0.8 large
Pooled SD		For equal group sizes

Probability

Quantity	Formula
Complement	
Addition	
Multiplication	
Independence	only if independent
Bayes' rule	
Expanded denominator	

Regression

Quantity	Formula	Units
Linear model		—
MAE		Original
RMSE		Original
R ²		None

RMSE ≥ MAE always. A large ratio means a few big errors dominate.

Classification

Quantity	Formula	Reads as
Logistic function		Maps any real to (0,1)
Log-odds		The model is linear in <i>this</i>
Odds		Not probability
Odds ratio		Multiplicative effect on odds
Log loss		Punishes confident errors
Precision		Of those flagged, how many were right
Recall		Of those to find, how many we found
Specificity		—
F1		Harmonic mean; near the smaller
Accuracy		Useless when classes are imbalanced

Other

Quantity	Formula
TF-IDF	
Euclidean distance	
Silhouette	per point, then averaged

Part 4 — Decision tables

Which summary statistic?

Situation	Use	Because
Roughly symmetric, no extremes	Mean and SD	Uses all the information
Skewed, or has extreme values	Median and IQR	Robust to the tail
You need a total	Mean	Total = mean $\times$ n
Comparing across different scales	z-scores	Removes units
Bimodal	None — report the groups separately	A single number describes nobody

Which chart?

Question	Form	Avoid
How is one variable distributed?	Histogram, density, box	Pie chart
How do a few categories compare?	Bar, zero baseline	3-D bar, donut
How do two numerics relate?	Scatter	Dual y-axes
How does something change over time?	Line	Bar for continuous time
How does a distribution differ by group?	Faceted histogram, violin	Many overlaid histograms
Composition of a whole?	Stacked bar, treemap	Pie beyond ~4 slices
More than six categories?	Small multiples, or highlight one	More colours

Which regression metric?

If	Use	Because
Being wrong by 20 is 10 $\times$ as bad as by 2	MAE	Errors count proportionally
Being wrong by 20 is catastrophically worse	RMSE	Squaring penalises the tail
You need a unitless comparison	R ²	Proportion of variance explained
You cannot say which	Stop	You have not defined the decision

Least squares already minimises squared error. If MAE is your real loss, ordinary regression is fitting the wrong objective.

Which classification metric?

Situation	Use	Not
Balanced classes, symmetric costs	Accuracy	—
Rare positive class	PR-AUC, recall at your threshold	Accuracy, ROC-AUC
Comparing models' ranking ability	ROC-AUC	Accuracy
Choosing an operating point	Precision and recall at each threshold	Any single number
Probabilities used directly	Calibration curve, log loss	Any threshold metric

Does this model need scaling?

Model	Scaling	Why
k-NN, k-means, SVM, PCA	Required	Distance is the model
Logistic / linear with regularisation	Required	Penalty is scale-dependent
Linear regression, no penalty	Optional	Only affects coefficient units
Decision tree, random forest	Not needed	Splits are within one feature

Which encoding?

Variable	Encoding	Note
Nominal, few levels	One-hot	Drop one level for OLS
Ordinal with meaningful spacing	Ordinal	Justify the spacing
Ordinal with unclear spacing	One-hot, or state the assumption	Likert scales live here
High-cardinality	Target encoding	Inside the pipeline, or it leaks

Which missingness response?

Mechanism	Can you detect it?	Response
MCAR	No, but plausible if no gradients	Dropping is unbiased, loses power
MAR	Look for gradients in observed variables	Model it; state the assumption
MNAR	No	No statistical fix. Sensitivity analysis and disclosure

Is this leakage?

Ask one question of every feature: **would I have this value, with this meaning, at the moment I need the prediction?**

Pattern	Verdict
Feature computed after the outcome occurred	Leakage
Scaler, imputer, or selector fitted before splitting	Leakage
Rolling window without <code>.shift(1)</code>	Leakage
Random split on time-ordered data	Leakage
Resampling before cross-validation split	Leakage
Feature known at prediction time, fitted inside a pipeline	Clean

Which claim can I make?

Claim	Needs	Example
Descriptive	A clean count and a correct unit	"The withdrawal rate was 11.5%"
Comparative	Counts plus uncertainty, and disaggregation	"Rates differ across programmes"
Predictive	Held-out evaluation and a baseline	"Week 1–4 submissions predict withdrawal"
Causal	Randomisation, or a very strong design	"Mentoring reduces withdrawal"

Part 5 — Every common mistake in one place

The same tables that appear in each chapter of Volume 1, collected here so you can scan them before submitting.

Chapter 1 — Thinking Like a Data Scientist

Mistake	Why it happens	How to catch it
Counting rows as people	<code>len()</code> is easy and always returns something	Say “one row is one ___” out loud before every count
Averaging an identifier	<code>describe()</code> includes it automatically	Set IDs to <code>str</code> on load, or drop them before describing
Claiming causation from correlation	The causal sentence is the interesting one	Label every claim’s rung before writing it down
Analysing before defining the target	“Disengaged” feels like it means something	Write the operational definition in Markdown first
Notebook works for you, fails for everyone else	Cells were run out of order	Runtime — Restart and run all before every submission

Chapter 2 — Collecting and Understanding Data

Mistake	Consequence	Prevention
Reporting a mean without a response rate	Overstates a self-selected group	Always report <code>n</code> , <code>N</code> , and who is missing
Merging without <code>validate=</code>	Silent row duplication	Use <code>validate=</code> on every merge
Reading IDs as integers	Lost leading zeros, meaningless means	<code>dtype={"student_id": str}</code>
Trusting a proxy because it correlates	Model learns the proxy, fails where they diverge	Write out the failure cases before using it
No data dictionary	Nobody can reuse or audit the work	Write it while you still remember

Chapter 3 — Describing Data: Centre, Spread, and Shape

Mistake	Why it matters	Prevention
Reporting a mean for bimodal data	Describes nobody	Always plot the distribution first
Mixing numpy and pandas SD	Silent 1–12% discrepancy	Set <code>ddof</code> explicitly, every time
“68% fall within one SD”	Only true for normal distributions	Check the shape before quoting the rule
Standardising after splitting train/test	Leakage — see Chapter 5	Fit the scaler on training data only

Mistake	Why it matters	Prevention
Treating a z-score as a fairness fix	It embeds the comparison group as a choice	State and defend the reference group

Chapter 4 — Data Quality and Cleaning

Mistake	Consequence	Prevention
<code>drop_duplicates()</code> on conflicting records	Silently keeps a random version	Sort by a timestamp and use <code>keep="last"</code>
Imputing before diagnosing the mechanism	Hides bias behind a complete-looking table	Diagnose first, impute later or not at all
Removing outliers by SD rule	Can delete an entire subpopulation	Ask “impossible or unusual?”
<code>errors="coerce"</code> without a follow-up count	Silently destroys unparseable data	Always count new NaNs after coercion
Cleaning in a notebook without a log	Nobody can audit or reproduce it	Write the log as you go

Chapter 5 — Transformation, Features, and Leakage

Mistake	Consequence	Prevention
Fitting a scaler before splitting	Optimistic scores	Put every transformer in a Pipeline
Ordinal-encoding a nominal variable	Model asserts a false order	One-hot, unless order is real
Feature selection outside cross-validation	Can score well on pure noise	<code>make_pipeline(SelectKBest(...), model)</code>
Using post-outcome features	Excellent scores, useless model	Ask: available at prediction time?
Centred rolling windows on time data	Uses the future	Trailing windows and <code>shift(1)</code>

Chapter 6 — Exploratory Data Analysis

Mistake	Consequence	Prevention
Quoting a mean without plotting	Describes a population that does not exist	Histogram first, always
Comparing aggregates across groups	Simpson’s paradox	Disaggregate by any composition-differing variable
Stratifying by everything	Every finding dissolves into noise	Name candidate confounders before looking
Reporting a correlation as an effect	Overclaims by two rungs	Write observation, interpretation, limitation

Mistake	Consequence	Prevention
Treating a small-cell reversal as a finding	Chases noise	separately Check n and the interval before interpreting

Chapter 7 — Visualization and Data Storytelling

Mistake	Consequence	Prevention
Truncated bar baseline	Visual difference unrelated to data	<code>ax.set_ylim(0, ...)</code> on every bar chart
Colour as the only channel	Chart fails for ~8% of male readers	Greyscale test before shipping
Unlabelled error bars	Ambiguous by a factor of ~10	State SD, SE, or CI in the caption
Dual y-axes	Any correlation can be manufactured	Use two panels
Descriptive titles	Reader must find the point themselves	State the finding, at the right rung

Chapter 8 — Probability and Bayesian Reasoning

Mistake	Consequence	Prevention
Reversing a conditional	Base-rate fallacy	Write both directions out in words
Ignoring the base rate	Massively overestimates precision on rare events	Always use natural frequencies
Assuming independence	Understates joint risk, often severely	Ask what could make events co-occur
Unseeded simulation	Irreproducible results	<code>default_rng(seed)</code> , always
Trusting a simulation's precision	Precision is not accuracy	Vary the model, not just n

Chapter 9 — Statistical Inference and Experiments

Mistake	Consequence	Prevention
Reporting p without an interval	Loses magnitude and units	Interval first, always
“Not significant” = “no effect”	Confuses absence of evidence with evidence of absence	Report what the interval rules out
Significance = importance	Any effect is significant at large n	State an MDE before testing
Peeking and stopping at $p < 0.05$	False-positive rate approaches 1	Fix n and the stopping rule in advance

Mistake	Consequence	Prevention
Many tests, no correction	~64% false positive across 20 tests	Pre-register the primary outcome

Chapter 10 — Regression and the Machine-Learning Workflow

Mistake	Consequence	Prevention
Reporting a score with no baseline	Number is uninterpretable	Always fit DummyRegressor first
Evaluating on training data	Overfitting invisible	Held-out set or cross-validation
Causal language for coefficients	Overclaims by two rungs	“Associated with”, and name what is held fixed
Choosing the metric after seeing results	Metric-shopping	State the metric before fitting
Reporting CV mean without SD	Hides instability	Always report the spread

Chapter 11 — Classification I: Building a Classifier

Mistake	Consequence	Prevention
Linear regression for a binary target	Predictions outside [0,1]	Logistic regression
Reporting accuracy on imbalanced data	88.6% means nothing here	Report against the majority baseline
Interpreting as a probability change	Off by a large factor	is an odds ratio
k-NN without scaling	One feature dominates distance	Always pipeline with a scaler
Unstratified split on a rare class	Train and test differ	<code>stratify=y</code>

Chapter 12 — Classification II: Errors, Thresholds, and Fairness

Mistake	Consequence	Prevention
Reporting accuracy on imbalanced data	Hides a useless model	Confusion matrix and PR-AUC
Optimising F1 by default	Assumes precision and recall matter equally	Derive weights from error costs
ROC-AUC on rare positives	Flatters the model	Precision-recall curve
Resampling before splitting	Leakage; inflated scores	Resample inside the fold
Reporting only aggregate metrics	Group failure invisible	Always disaggregate
Per-group metrics on tiny	False precision	Report n and an interval, or

Mistake	Consequence	Prevention
cells		decline to report

Chapter 13 — Clustering and Dimensionality Reduction

Mistake	Consequence	Prevention
Clustering unscaled features	Widest-range feature decides everything	<code>StandardScaler</code> in a pipeline
Treating k as discoverable	Overstates structure that may not exist	Justify k by use; run a null test
Interpretive cluster names	Smuggles unsupported claims about people	Name from features only
Not checking seed stability	Names artefacts of initialisation	Refit across seeds, compare with ARI
Interpreting PCs as causes	Components are variance directions	Report loadings; hold names loosely

Chapter 14 — Time Series and Text

Mistake	Consequence	Prevention
Random split on time data	Trains on the future	Chronological split or <code>TimeSeriesSplit</code>
Rolling window without <code>shift(1)</code>	Includes the current value	<code>.shift(1).rolling(n)</code>
No seasonal-naive baseline	Model looks good against nothing	Always compute it first
Fitting the vectorizer before splitting	Vocabulary leakage	Vectorizer inside the pipeline
Treating text accuracy as sentiment truth	Negation and sarcasm invisible	Read the misclassified examples

Chapter 15 — Dashboards, Delivery, and Responsible Practice

Mistake	Consequence	Prevention
Suppressing one cell per row	Recoverable by subtraction	Complementary suppression
Rounding as privacy protection	Differencing still works	Suppress, do not round
Dashboard with no attached action	Maintenance cost, no value	State the decision per view
Limitations after conclusions	Conclusions get written too strong	Write limitations first
A file path only you have	Nobody else can run it	Define one <code>DATA = ...</code> variable at the top and use it everywhere

Mistake	Consequence	Prevention
Deploying without a control group	Cannot tell whether it worked	Randomise a holdout

NUU · Fundamentals of Data Science · Student Copy

Part 6 — Glossary

Term	Meaning
Accuracy	Proportion of predictions that are correct. Misleading when classes are imbalanced.
Algorithm	A defined procedure for producing an output from inputs.
Autocorrelation	Correlation of a series with its own past values.
Baseline	A trivial reference model. Without one, a score is uninterpretable.
Base rate	The unconditional frequency of an outcome. Dominates the interpretation of rare-event tests.
Bias	A systematic distortion; in responsible AI, also harmful social patterns reproduced by a model.
Calibration	Whether predicted probabilities match observed frequencies.
Confounder	A variable associated with both the supposed cause and the outcome.
Cross-validation	Repeated train/test splitting to estimate generalisation.
Data leakage	Use of information unavailable at prediction time.
Data dictionary	Column-level documentation of type, meaning, units, and permitted values.
Effect size	Magnitude of a difference, independent of sample size.
Feature	An input variable used by a model.
Generalisation	Performance on relevant unseen data.
Imputation	Replacing missing values with estimates. Assumes a missingness mechanism.
Log-odds	. Logistic regression is linear in this.
MAR / MCAR / MNAR	Missingness mechanisms; only MNAR has no statistical remedy.
Model	A computational representation fitted to data.
Odds ratio	. Multiplicative effect on odds, not on probability.
Operational definition	The rule turning a concept into a measurable column.

Term	Meaning
Overfitting	Learning training noise; excellent training and poor test performance.
Parameter	A quantity describing a population or a learned model.
Power	Probability of detecting an effect that exists.
Precision	Of those flagged, the proportion correct.
Provenance	The documented origin and history of data.
Proxy	An observable variable standing in for an unmeasured concept.
p-value	. Not the probability the null is true.
Recall	Of those that should be found, the proportion found.
Reproducibility	The ability to repeat an analysis from documented inputs and procedures.
Residual	Observed minus predicted.
ROC-AUC	Probability a random positive is ranked above a random negative.
Sampling frame	The list from which a sample is actually drawn.
Silhouette	Cluster separation measure, -1 to 1. Above 0.5 indicates reasonable structure.
Simpson's paradox	An association that reverses when a composition variable is controlled.
Standard error	Standard deviation of a sampling distribution.
Stratification	Analysing within levels of a variable to check whether a relationship holds.
Target	The outcome a supervised model predicts.
TF-IDF	Term weighting by document frequency and corpus rarity.
Threshold	The probability cut converting a score into a decision. A policy choice.
Uncertainty	Lack of complete knowledge about a quantity, event, or conclusion.
Unit of observation	What one row represents. Determines which summaries are valid.

Part 7 — The dataset

The tables

File	Rows	One row is
students.csv	1,200	one student
enrolments.csv	4,665	one student-course registration
activity.csv	16,165	one student-week of platform activity
outcomes.csv	1,200	one student, with weeks 1–4 features and the target
feedback.csv	732	one free-text course comment
platform_daily.csv	364	one day of platform sessions
students_dirty.csv	1,205	students.csv with realistic damage injected

The tables are linked but **not pre-joined**. Joining them is part of the work, and joining them carelessly will duplicate rows.

students.csv

Column	Type	Meaning
student_id	integer	Unique. Nominal despite being numeric — never average it.
programme	string	4 levels. Nominal.
entry_cohort	integer	2024–2026
entry_score	float	Entry qualification, 0–100. Continuous.
study_mode	string	Full-time / Part-time
age_at_entry	integer	Whole years. Discrete.
first_generation	boolean	Sensitive . See the note below.

enrolments.csv

Column	Type	Meaning
student_id	integer	Foreign key
course_code	string	12 courses
term	string	e.g. 2025-T3
final_grade	float	0–100
passed	boolean	final_grade >= 50

activity.csv

Column	Type	Meaning
student_id	integer	Foreign key
iso_week	integer	Teaching week, 1–14
logins	integer	Distinct logins that week
submissions	integer	Assessment submissions, 0–2
forum_posts	integer	Discussion posts
minutes_on_platform	integer	Total minutes

Rows after a student withdraws are **absent**, not zero. Students therefore have different numbers of weeks, which is what makes full-term activity a leaky feature for predicting withdrawal.

outcomes.csv

Column	Type	Meaning
student_id	integer	Foreign key
early_logins	integer	Logins summed over weeks 1–4
early_submissions	integer	Submissions summed over weeks 1–4
early_minutes	integer	Minutes summed over weeks 1–4
midterm_satisfaction	float	1–5. 23.7% missing, not at random.
withdrew	boolean	Target. 11.5% positive.
withdrew_week	float	Week of withdrawal, 5–13. Do not use as a feature.

feedback.csv and platform_daily.csv

feedback.csv: student_id, course_code, comment (free text), sentiment (positive / negative).

platform_daily.csv: date (ISO), sessions (integer). One row per day for 364 days, with trend, weekly seasonality, and a term-break level shift.

students_dirty.csv

students.csv with damage injected for the cleaning chapter: mixed case and stray whitespace in programme (16 variants for 4 programmes), nine impossible values in age_at_entry, 44 entry_score values stored as text with a % suffix, missing values in three columns, a registered_on column in three date formats, two exact duplicate rows, and three duplicate-with-update pairs.

Provenance and licence

Synthetic, generated with a fixed seed. Released under **CC0** — copy, modify and redistribute freely. **Contains no real student record.**

That last point is deliberate. This course argues repeatedly that a technically possible analysis of data about people is not automatically a legitimate one, and it would be inconsistent to make that argument on top of real records.

Note on sensitive attributes

`first_generation`, `age_at_entry`, and `study_mode` are all attributes that could be used to disadvantage a student. The course position is that these are legitimate **for auditing whether outcomes differ across groups** and illegitimate **as features in a model that allocates or withholds something**. If your model uses one as an input, say so explicitly and justify it.

NUU · Fundamentals of Data Science · Student Copy

Part 8 — Responsible-practice checklist

Framing. Who benefits and who bears risk? Is a less intrusive analysis sufficient? What action follows, and what oversight applies? Is the claim type (descriptive, comparative, predictive, causal) stated?

Data. Source, licence, and consent recorded? Unit of observation stated? Coverage gaps identified? Sensitive attributes identified and their use justified?

Analysis. Every feature available at prediction time? Preprocessing inside a pipeline? Baseline computed? Performance disaggregated by group? Small cells identified?

Delivery. Limitations as prominent as findings? Uncertainty shown? Cells suppressed with a complementary rule? Affected people able to learn of and contest decisions? Recommendation proportionate? Notebook runs top to bottom from clean, checked by someone else?

NUU · Fundamentals of Data Science · Student Copy

Part 9 — Companion books and further study

The two companion books

You do not need to buy anything. Both books below are **free and legal to download**, both use the same tools this course uses, and between them they cover the two halves of the syllabus at greater depth.

Companion A — for Chapters 3, 8, and 9

OpenIntro Statistics (4th edition) Diez, Çetinkaya-Rundel & Barr

<https://www.openintro.org/book/os/> — free PDF, print edition available cheaply

This is the gentlest good book on the statistics this course needs. Where our Chapters 3, 8 and 9 give you the working knowledge, OpenIntro gives you the surrounding theory at a pace designed for readers without a mathematics background.

Our chapter	Read there
3 — Describing data	Ch 1–2: data basics, summarising
6 — Exploratory analysis	Ch 2: exploring categorical and numerical data
8 — Probability and Bayes	Ch 3: probability, conditional probability, Bayes
9 — Inference and experiments	Ch 5–7: sampling distributions, CIs, tests

It also has extensive exercises with solutions, which is the main reason to have it: more practice at the same level, when four questions per chapter is not enough.

One difference to expect. OpenIntro's examples are in R, and some editions have a Python companion. The statistical reasoning transfers exactly; only the syntax differs, and you already have the syntax in Part 2 of this volume.

Companion B — for Chapters 10 to 13

An Introduction to Statistical Learning, with Applications in Python (ISLP) James, Witten, Hastie, Tibshirani & Taylor (2023) <https://www.statlearning.com/> — free PDF, with labs and datasets

This is the standard first book on statistical learning, and the Python edition uses scikit-learn much as we do. Where our Chapters 10–13 teach you to build and evaluate a model responsibly, ISLP explains what the models are doing underneath.

Our chapter	Read there
10 — Regression and the ML workflow	Ch 2–3: assessing model accuracy, linear regression
11 — Building a classifier	Ch 4: logistic regression, LDA, k-NN
10 — Cross-validation	Ch 5: resampling methods

Our chapter	Read there
11 — Trees	Ch 8: trees, bagging, random forests
13 — Clustering and PCA	Ch 12: unsupervised learning

A warning about ISLP. It is more mathematical than this course, and Chapter 6 onward assumes comfort with matrix notation. Read Chapters 2–5 and 12 first; they are the ones that map onto our material. Do not be discouraged by the parts that are harder than you need — they are for a later course.

If you can have only one

Take **ISLP**. The statistics in Chapters 3, 8 and 9 are covered well enough here for this course's purposes, but the modelling chapters are where you will most want a second explanation, and ISLP is the best free one.

Specialist references, for one week each

Topic	Book	Free?
pandas, in depth	McKinney, <i>Python for Data Analysis</i> (3rd ed) — wesmckinney.com/book	Yes, online
Time series (Ch 14)	Hyndman & Athanasopoulos, <i>Forecasting: Principles and Practice</i> (3rd ed) — otexts.com/fpp3	Yes
Fairness (Ch 12, 15)	Barocas, Hardt & Narayanan, <i>Fairness and Machine Learning</i> — fairmlbook.org	Yes
Visualization (Ch 7)	Wilke, <i>Fundamentals of Data Visualization</i> — clauswilke.com/dataviz	Yes
Regression, deeper (Ch 10)	Gelman, Hill & Vehtari, <i>Regression and Other Stories</i>	No

Full reference list

Adhikari, A., DeNero, J., & Wagner, D. (2022). *Computational and Inferential Thinking: The Foundations of Data Science* (2nd ed.). UC Berkeley Data 8. <https://inferentialthinking.com/>

ACM. (2018). *ACM Code of Ethics and Professional Conduct*. <https://www.acm.org/code-of-ethics>

Barocas, S., Hardt, M., & Narayanan, A. (2023). *Fairness and Machine Learning: Limitations and Opportunities*. MIT Press. <https://fairmlbook.org/>

Diez, D., Çetinkaya-Rundel, M., & Barr, C. (2019). *OpenIntro Statistics* (4th ed.). <https://www.openintro.org/book/os/>

Downey, A. B. (2014). *Think Stats: Exploratory Data Analysis in Python* (2nd ed.). O'Reilly Media.

Gelman, A., Hill, J., & Vehtari, A. (2020). *Regression and Other Stories*. Cambridge University Press.

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning: With Applications in Python*. Springer. <https://www.statlearning.com/>

McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media. <https://wesmckinney.com/book/>

NIST. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1. <https://doi.org/10.6028/NIST.AI.100-1>

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Wilke, C. O. (2019). *Fundamentals of Data Visualization*. O'Reilly Media. <https://clauswilke.com/dataviz/>

Wilkinson, M. D., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018.

Source and adaptation note

For deeper study, compare this book with UC Berkeley Data 8, Harvard CS109, and MIT OpenCourseWare 6.0002. The statistics chapters were cross-checked against OpenIntro Statistics; the modelling chapters against ISLP; the forecasting chapter against Hyndman & Athanasopoulos; the fairness material against Barocas, Hardt & Narayanan; and responsible-practice content draws on the ACM Code of Ethics and the NIST AI Risk Management Framework. No passage, exercise, guided project, table, or figure was copied from those sources.

All datasets are synthetic and generated by `make_dataset.py` in the accompanying materials. All figures are generated by `make_figures.py`. Every numerical result quoted in the text was computed from those datasets and verified by `test_book_code.py`.

Verify before printing. Edition numbers, chapter numbering and URLs change between printings. Check each link and chapter mapping above against the current edition before putting this in front of students.